



I E S A L B A R R E G A S

TÉCNICO SUPERIOR EN DESARROLLO DE
APLICACIONES WEB

Departamento de Informática

PROYECTO

Lucina Comparador Eléctrico con IA
Manual Técnico

Autor: Javier González Paniagua
Curso Académico: DAW 2025 - 2026

ÍNDICE

1 - Introducción	3
2 - Arquitectura de la aplicación	7
2.1 Frontend	8
2.1.1 Tecnologías usadas	9
2.1.2 Entorno de desarrollo	10
2.2 Backend	10
2.2.1 Tecnologías usadas.....	10
2.2.2 Entorno de desarrollo	11
3 - Documentación técnica	11
3.1 Análisis.....	11
3.2 Desarrollo (diagrama de secuencias).....	12
3.2.1 Algoritmo de Comparativa y Ahorro IA	13
3.2.2 Sistema de Notificaciones "Email" (Sockets).....	13
3.2.3 Seguridad y Gestión de Archivos	13
3.2.4 Modelo de la Base de datos:	14
3.3 Pruebas realizadas	15
3.3.1 Prueba de Extracción Inteligente (IA)	15
3.3.2. Prueba del Algoritmo de Ahorro	15
3.3.3. Prueba de Notificaciones por Sockets	15
3.3.4. Prueba de Seguridad (Inyección SQL)	16
4 - Proceso de despliegue	16
4.1 Entorno de Desarrollo (Local)	16
4.2. Entorno de Producción (Hosting Real).....	17
5 - Propuestas de mejoras.....	17
5.1 Seguridad y protección del servidor	17
5.2 Área de cliente y autocontratación	18
5.3 Infraestructura y gestión.....	18
5.4 Aviso Legal y RGPD.....	18
6 Bibliografía	19
6.1 Documentación Técnica y Manuales	19
6.2 Recursos Web y Frameworks	19

1 - Introducción



El proyecto Lucina es fruto de una necesidad de software que disponen las empresas que gestionan contratos de Energía e incluso de los propios usuarios para poder solucionar un problema que mucha gente tiene que es “Comprender la Factura de la Luz” este software permite aprovechar la Inteligencia Artificial con la API de Gemini en este caso, para ahorrar tiempos y costes.

El objetivo principal ha sido eliminar el cuello de botella que supone la entrada de datos manual de la factura de cliente. Para ello se ha implementado un extractor OCR basado en Google Gemini que permite transformar un documento no estructurado (PDF/Imagen) en datos técnicos. Esta aproximación reduce el error humano casi a cero y permite escalabilidad del negocio antes impensable. Como por ejemplo que el propio cliente pueda subir la factura con una Landing Page y el único trabajo del comercial sea asistir al cliente o cerrar la contratación.

Principales Funciones:

Motor de Extracción Inteligente (OCR IA): Integración avanzada con API de Gemini para el procesamiento de documentos, configurada como un prompt de ingeniería que garantiza la extracción de consumos, potencias, CUPS en formato JSON validado.

Sistema de Comparativa y Ahorro: Algoritmo de cálculo desarrollado en PHP que cruza datos extraídos con el catálogo de tarifas vigentes en el sistema para ofrecer una comparativa de ahorro anual instantánea con los productos que comercializamos.

Modulo de Mantenimiento de Tarifas: Interfaz completa para que el administrador pueda actualizar comercializadoras, precios de energía, potencias, presupuestos y clientes.



LUCINA ADMIN

RESUMEN

Escritorio

Compañías

Catálogo de Tarifas

CRM CLIENTES

Presupuestos

Cerrar sesión

LUCINA ADMIN

RESUMEN

Escritorio

Compañías

Catálogo de Tarifas

CRM CLIENTES

Presupuestos

Cerrar sesión

LUCINA ADMIN

RESUMEN

Escritorio

Compañías

Catálogo de Tarifas

CRM CLIENTES

Presupuestos

Cerrar sesión

Escritorio Principal

Sesión: Administrador

3 Clientes Totales

2 Compañías Activas

4 Tarifas Publicadas

3 Análisis Realizados

Últimas Solicitudes de Ahorro

Ver gestión completa →

CLIENTE	CONTACTO PRINCIPAL	ÚLTIMA ACTIVIDAD	ESTADO	ACCIONES
Javier Fatela	✉ javierfatela@lucina.es	12/04/2026 21:29	Activo	🔍
javier	✉ javier@lucina.es	12/04/2026 21:23	Activo	🔍
Eva	✉ eva@lucina.es	12/04/2026 21:22	Activo	🔍

Compañías Eléctricas

+ Registrar Nueva

Cartera de Proveedores

2 empresas

NOMBRE COMERCIAL	IMAGEN DE MARCA	VIGENCIA	ACCIONES
Gana Energía	💡 Gana Energía	Operativa	✎ 🗑
Naturgy	Naturgy	Operativa	✎ 🗑

Gestión de Tarifas

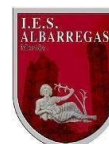
+ Crear Nueva Tarifa

Tarifas de Luz Publicadas

4 registros

COMERCIALIZADORA	NOMBRE DE LA OFERTA	EENERGÍA (P1)	ENERGÍA (P2)	ENERGÍA (P3)	ESTADO	ACCIONES
Gana Energía	Tarifa 24 Horas	0.1290 €	0.1290 €	0.1290 €	Activa	✎ 🗑
Gana Energía	Tarifa Tramos Horarios	0.1810 €	0.1140 €	0.0900 €	Activa	✎ 🗑
Naturgy	Tarifa Noche	0.1802 €	0.1072 €	0.0718 €	Activa	✎ 🗑
Naturgy	Tarifa Por Uso Luz	0.1099 €	0.1099 €	0.1099 €	Activa	✎ 🗑

Proyecto “Desarrollo de Aplicaciones Web”



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

LUCINA ADMIN

GESTION GENERAL

Escritorio

Compañías

Catálogo Tarifas

MANEJO DE CLIENTES

Listado Clientes

Cerrar Sesión

Información Personal

JF

Javier Fatela

Estado: Activo

Correo: javierfatela@lucina.es

Móvil: 666123789

Población: PLASENCIA

Notas Internas

Sin notas registradas.

Historial de Estudios Energéticos

1 Registro

CUPS: [ES0021000012961672YL](#)

Fecha de carga: 12/04/2026 21:29

Coste Actual **89.01 €**

Punta (P1)
82.00 kWh

Llano (P2)
118.00 kWh

Valle (P3)
180.00 kWh

Potencia P1
3.45 kW

Comparativa Directa por Tarifas del Sistema

COMPANÍA / TARIFA	COSTE ESTIMADO	DIFERENCIA
Gana Energía / Tarifa 24 Horas	86.87 €	+2.14 €
Gana Energía / Tarifa Tramos Horarios	81.12 €	+7.89 €
Naturgy / Tarifa Por Uso Luz	75.21 €	+13.80 €
Naturgy / Tarifa Noche	73.41 €	+15.60 €

Original

Editar

Borrar

* Proyección basada en tarifas del catálogo.

Landing Page Pública: preparada para ser compartida vía QR, Link, o campañas de marketing donde en 2 sencillos pasos obtenemos los datos del cliente y le facilitamos su comparativa.



1
ANÁLISIS

2
VALIDACIÓN

3
AHORRO

Optimiza tu factura eléctrica y **ahorra**

Sube tu factura ahora para que nuestro sistema la analice y encuentre la mejor tarifa para ti.



Análisis con IA de Gemini

Tecnología de última generación para procesar tus PDF de factura al instante.

1. Identifícate

Nombre Completo

Ej: Juan Pérez

Correo Electrónico

juan@correo.com

Teléfono de contacto

666 000 000

2. Adjunta tu última factura



Suelte aquí su factura (PDF o imagen)

O haga clic para explorar sus archivos

Máxima confidencialidad en el tratamiento de datos

Seguro

Rápido

Sin compromisos

2 - Arquitectura de la aplicación

La arquitectura de Lucina se ha diseñado bajo un enfoque de Monolito Modular Organizado con separación lógica de responsabilidades por las siguientes razones:

Eficiencia y Rapidez de Despliegue: El Objetivo es la entrega de una solución funcional y estable.

Mantenibilidad Modular: Aunque el núcleo es único, el código se ha dividido en módulos independientes (Análisis IA, Gestión de datos, Notificaciones por Sockets). Esto permite que cada componente sea escalable o sustituible en el futuro sin comprometer el resto del sistema.

Reducción de Latencia: Al procesar las solicitudes de IA y los cálculos de ahorro en el mismo servidor donde reside la lógica de negocio, eliminamos saltos de red innecesarios, ofreciendo una respuesta inmediata al usuario final.

Enfoque Senior en seguridad: La arquitectura no delega la seguridad en servicios de terceros, si no que implementa su propia capa de validación mediante consultas preparadas (PDO) y unas gestiones de sesiones robusta desde el servidor.

Esta elección no es solo una simplificación técnica, sino una decisión estratégica dado que permite que Lucina sea una herramienta “plug & play”, fácil de instalar en la infraestructura de cualquier consultoría energética sin necesidades complejas ya que solo necesita un servidor web Apache con php 8.0 o superior, MySQL o MariaDB y una clave API de Gemini.

2.1 Frontend

Esta capa se encarga de la interfaz de usuario, la experiencia de navegación (UX) y el procesamiento previo de los archivos cargados por el cliente.

¿Estás pagando de más en tu factura de la luz?

Sube una foto o el PDF de tu última factura y te buscamos la mejor tarifa ahora mismo.

ARRASTRA TU FACTURA AQUÍ

SELECCIONAR ARCHIVO

Tus datos están seguros con nosotros, solo los usamos para el cálculo.

DIBUJO LUPA

Análisis real

Leemos los datos de tu contador y buscamos entre todas las compañías.

DIBUJO BILLETES

Ahorro directo

Si hay una tarifa más barata, te lo decimos sin vueltas.

DIBUJO RELOJ

En un minuto

No pierdas tiempo comparando tú mismo, nuestro sistema lo hace solo.

Presupuesto Final Cliente

LUCINA

Propuesta #1234
Fecha: 19/02/2026

Tus datos

Titular: [Juan Pérez]
Dirección: [C/ Ejemplo, 123]
Tu CUPS: [ES002...]

Lo que pagas ahora

Estás con: [Iberdrola / Endesa...]
Tu tarifa: [Nombre de la tarifa]
Al mes pagas unos: 95,00 €

Nuestra comparativa para ti

TABLA DE LOGOS DE COMPAÑÍAS

Concepto	Ahora pagas	Con nuestra opción	Lo que te ahorras
Potencia (término fijo)	15,00 €	12,00 €	3,00 €
Energía (lo que gastas)	80,00 €	65,00 €	15,00 €
TOTAL AL MES (ESTIMADO)	95,00 €	77,00 €	18,00 €

Cambiándote podrías ahorrar al año:

216,00 €

SÍ, ME INTERESA EL CAMBIO

2.1.1 Tecnologías usadas

Bootstrap V5: Se ha elegido este framework para la maquetación responsiva. La justificación técnica es la madurez y sistema de utilidades CSS, que permite un diseño profesional y adaptado a dispositivos móviles sin sobrecargar el peso de la página.

Vanilla Javascript: Se ha evitado el uso de librerías como jQuery o frameworks pesados como React para mantener un rendimiento óptimo, se utiliza en la zona de carga de archivos (File API) y las peticiones asíncronas del motor de análisis.

CSS3: Uso de variables nativas para un mantenimiento centralizado de la identidad visual.

2.1.2 Entorno de desarrollo

Visual Studio Code: Es el editor principal. Se justifica por su ecosistema de extensiones para la validación de sintaxis HTML/CSS en tiempo real.

Chrome DevTools: Herramienta fundamental para el desarrollo de la interfaz, utilizada para la depuración del DOM, auditorías de rendimiento y pruebas de respuesta en diferentes resoluciones de pantalla.

Xamp: Para el entorno de pruebas se ha instalado en el equipo un servidor tipo Portable con XAMP que facilita la instalación en 1 clic de un servidor local de Apache, PHP, MySQL y PHPMyAdmin.

2.2 Backend

El motor de Lucina, encargado del análisis de datos mediante IA, la persistencia en la base de datos y el algoritmo de comparativa energética.

2.2.1 Tecnologías usadas

PHP v8.2.x: Es el núcleo de la aplicación. Su elección se justifica por su excelente rendimiento en la versión 8 y su capacidad nativa para manejar flujos de datos cURL necesarios para la comunicación con la API de Google.

MariaDB v10.4.x (Motor InnoDB): La base de datos relacional del proyecto. Se ha optado por el motor InnoDB para asegurar la integridad referencial de los datos de facturación y tarifas mediante claves foráneas.

API de Google Gemini (1.5 Flash): El "cerebro" de extracción. Se justifica por su capacidad multimodal para entender facturas complejas de forma semántica, algo que un OCR tradicional no puede lograr.

SMTP por Raw Sockets: Una implementación propia (clase LucinaCorreo) para el envío de notificaciones. Se justifica para demostrar capacidad técnica en protocolos de red y evitar dependencias de terceros.

2.2.2 Entorno de desarrollo

XAMPP v3.3.0: Entorno que integra Apache, PHP y MySQL. Se ha elegido por su capacidad de replicar fielmente un entorno de servidor de producción en local, permitiendo pruebas integrales rápidas.

Postman: Utilizado intensivamente durante la fase de desarrollo para validar los endpoints de la IA de Google y asegurar que los objetos JSON recibidos coinciden con la estructura esperada por nuestra base de datos.

PDO (PHP Data Objects): Capa de acceso a datos que garantiza la seguridad del sistema mediante el uso de sentencias preparadas para mitigar cualquier riesgo de inyección SQL.

3 - Documentación técnica

3.1 Análisis

En esta fase se ha definido la estructura lógica y funcional del sistema, asegurando que la base de datos y los procesos de negocio estén alineados con los objetivos de Lucina.

El sistema identifica dos actores principales con roles claramente diferenciados:

Cliente / Usuario Final: Es el actor que interactúa con la Landing page. Sus casos de uso incluyen: Subir Factura, Consultar Estudio de Ahorro y Notificar Interés Comercial.

Administrador: Perfil técnico que gestiona el Backend. Sus casos de uso incluyen: Mantener Catálogo de Tarifas, Gestionar Compañías Energéticas y Supervisar Leads.

Mapa de Sitio: Arquitectura Lucina

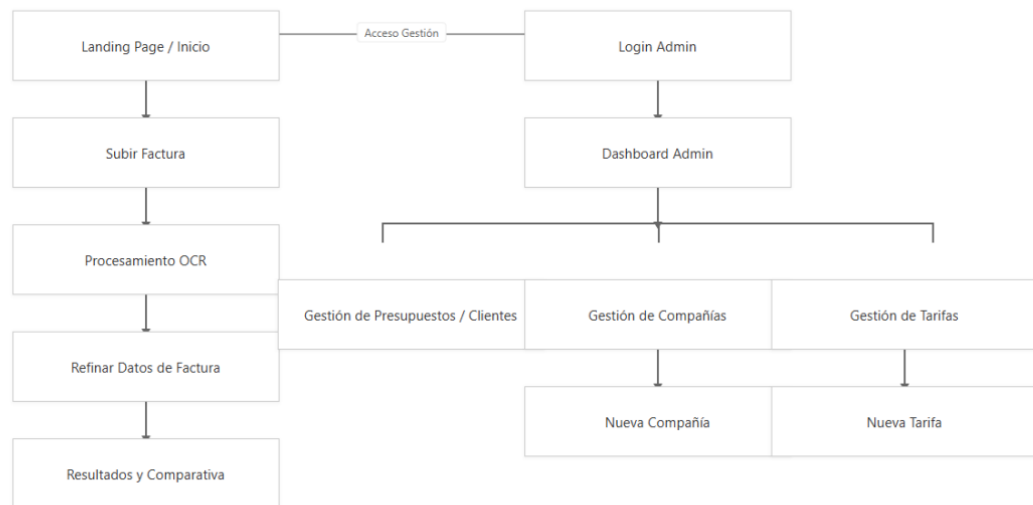


Diagrama Entidad-Relación (E-R)

Se ha diseñado un modelo relacional normalizado para garantizar la integridad de los datos. La tabla clientes actúa como núcleo, vinculada a las facturas procesadas. Las tablas tarifas y compañías forman el catálogo maestro, permitiendo una relación 1:N que facilita la actualización masiva de precios sin redundancia.

3.2 Desarrollo (diagrama de secuencias)

Desarrollo Técnico y Lógica de Negocio

El desarrollo de Lucina ha seguido una metodología centrada en la robustez del dato y la eficiencia en el procesamiento asíncrono. A continuación se detallan los núcleos lógicos del sistema:

Para el desarrollo técnico de Lucina, me he centrado en solucionar los problemas reales que surgen al intentar automatizar el sector energético. He dividido la lógica en varios bloques que se comunican entre sí de forma fluida.

3.2.1 Algoritmo de Comparativa y Ahorro IA

La lógica de comparación está centralizada en ahorro.php mediante la función `calcularCosteTeorico()`. Aquí es donde aplico los impuestos reales del sistema eléctrico español, como el Impuesto Eléctrico (5,11%) y el IVA (21%). De esta forma, el ahorro que ve el usuario no es una estimación genérica, sino un cálculo matemático real basado en sus consumos de periodos P1, P2 y P3 extraídos de su propia factura.

3.2.2 Sistema de Notificaciones "Email" (Sockets)

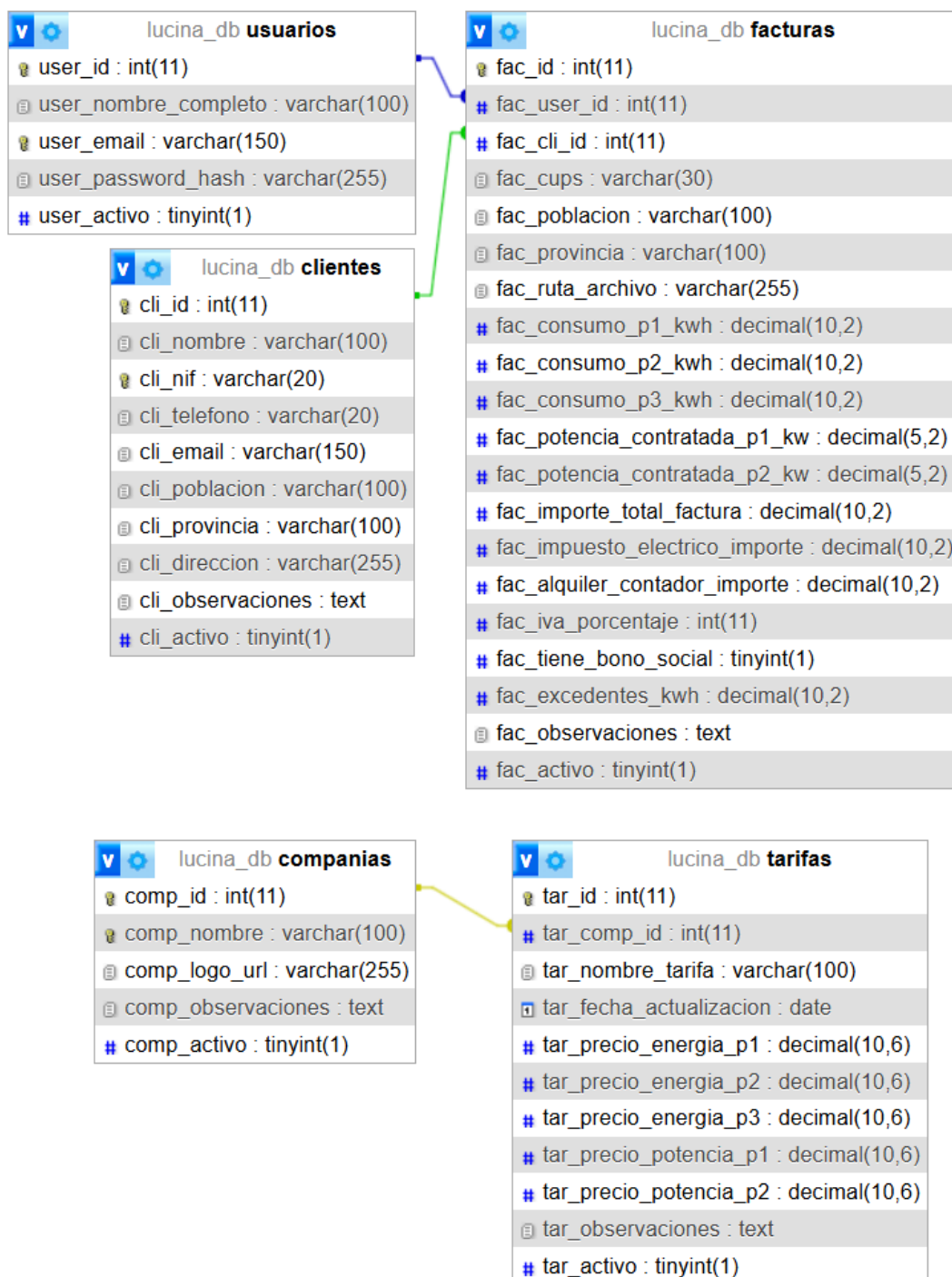
Para las notificaciones de interés, quise evitar librerías externas que suelen dar problemas de versiones o dependencias. Por eso, en `funciones/notificaciones.php`, programé una clase propia con Sockets. Me conecto directamente al servidor SMTP mediante `fsockopen` y gestiono manualmente los comandos de negociación (EHLO, AUTH LOGIN, DATA).

Esto me permite tener un control total sobre el envío de correos sin añadir peso extra al proyecto.

3.2.3 Seguridad y Gestión de Archivos

La seguridad ha sido una prioridad constante. Todas las consultas a MariaDB las realizo mediante sentencias preparadas con PDO para evitar inyecciones SQL. Además, para que los archivos subidos no se pisen entre ellos ni sean predecibles, les cambio el nombre usando un hash MD5 único en el momento de guardarlos en la carpeta `/archivos`.

3.2.4 Modelo de la Base de datos:



3.3 Pruebas realizadas

3.3.1 Prueba de Extracción Inteligente (IA)

Componente: Script de procesamiento funciones/analizar.php y conexión con la API de Google Gemini.

Juego de datos de entrada: Facturas reales en formato PDF de varias comercializadoras (con CUPS visible y consumos en los tres periodos).

Resultado de salida: Objeto JSON validado con los campos cups, consumo_p1, consumo_p2 y consumo_p3 mapeados correctamente con sus valores numéricos.

Herramientas de pruebas usadas: Postman (para interceptar la respuesta de la API) y Chrome DevTools (para verificar la recepción del JSON en el Frontend).

3.3.2. Prueba del Algoritmo de Ahorro

Componente: Función calcularCosteTeorico() en el archivo ahorro.php.

Juego de datos de entrada: Consumos fijos de prueba (100 kWh en P1, 50 kWh en P2) cruzados con una de las tarifas almacenadas en la base de datos (Precio P1: 0.15€, Precio P2: 0.12€).

Resultado de salida: Cálculo matemático exacto sumando Energía + Término de Potencia + Impuesto Eléctrico (5,11%) + IVA (21%). El sistema devolvió 29.56€, coincidiendo con el cálculo manual realizado en Excel.

Herramientas de pruebas usadas: Excel y Gemini Chat para depurar paso a paso el cálculo de la variable \$totalFactura.

3.3.3. Prueba de Notificaciones por Sockets

Componente: Clase enviar() dentro de funciones/notificaciones.php.

Juego de datos de entrada: Intento de aviso al administrador con los datos de un cliente interesado (Nombre: "Juan Prueba", Email: "test@lucina.es").

Resultado de salida: Registro de sesión SMTP exitoso. El servidor devolvió los códigos de estado 220 (Conexión), 250 (Aceptación de datos) y 221 (Cierre de socket), confirmando la recepción del correo.

Herramientas de pruebas usadas: Monitorización de logs de servidor y correo electrónico.

3.3.4. Prueba de Seguridad (Inyección SQL)

Componente: Módulo de autenticación del panel de administración admin/login.php.

Juego de datos de entrada: Intento de acceso usando en el campo de usuario la cadena: ' OR 1=1 --.

Resultado de salida: Acceso denegado. El sistema redirigió correctamente a la página de error de login, demostrando que el uso de PDO y sentencias preparadas mitiga este tipo de ataques.

Herramientas de pruebas usadas: Pruebas de penetración manuales sobre los formularios de entrada.

4 - Proceso de despliegue

He estructurado el proyecto para que sea lo más "plug & play" posible. A continuación se detallan los pasos para los dos entornos principales utilizados:

4.1 Entorno de Desarrollo (Local)

Preparación: Uso de XAMPP v3.3.0 con PHP 8.2 y MariaDB.

Base de Datos: Importación del archivo lucina_db.sql desde phpMyAdmin local.

Configuración: Edición de config.php con las credenciales de local (localhost).

4.2. Entorno de Producción (Hosting Real)

Para la puesta en marcha real del proyecto, se ha realizado un despliegue en un entorno de producción profesional:

Hosting: Servidor gestionado en Bana Hosting bajo el dominio corporativo de la empresa familiar.

URL de Producción: <https://maskfundas.es/lucina/>

Método de Despliegue: Creación de carpeta específica y transferencia de archivos mediante protocolos seguros de FTP.

Base de Datos: Creación de una nueva base de datos MariaDB en el panel de Bana hosting e importación del esquema completo desde PHPMyAdmin.

Seguridad SSL: Configuración de certificado de seguridad para garantizar que la carga de documentos PDF se realice de forma cifrada mediante HTTPS.

5 - Propuestas de mejoras

De cara al futuro, tengo pensadas varias mejoras para que Lucina sea una herramienta mucho más completa y segura. Estas son las ideas principales que me gustaría implementar:

5.1 Seguridad y protección del servidor

Lo primero que quiero hacer es reforzar la seguridad para que no nos tumben la web:

Poner el Captcha de Google: Lo añadiría tanto en la página donde se suben las facturas como en el login del administrador, para evitar que entren bots a dar la lata.

Sistema de autoban: Si alguien intenta entrar al panel de administración y falla la contraseña 3 veces, el sistema bloqueará su IP automáticamente.

Limitar las subidas por IP: Para que el servidor no sufra ataques y problemas de rendimiento, quiero limitar el número de estudios que puede pedir una misma persona (IP) en una hora.

5.2 Área de cliente y autocontratación

Quiero que el usuario tenga más protagonismo y pueda hacer más cosas solo:

Panel de usuario: Me gustaría que los clientes pudieran registrarse para ver todos los estudios que han hecho y saber cuándo les toca renovar el contrato para volver a comparar precios.

Que el usuario contrate solo: La idea es que, si le gusta una oferta, el sistema le pida todos los datos (IBAN, dirección completa...) y genere el contrato para que pueda firmarlo digitalmente en el momento.

Conectividad y datos reales

Para que el ahorro sea exacto al 100%:

Conexión con DATADIS: Una vez que la IA nos da el CUPS, quiero que el sistema se conecte con las distribuidoras para bajar el consumo real de ese cliente.

5.3 Infraestructura y gestión

Registro Dominios y VPS: Registrar lucina.com y lucina.es y apuntarlos directamente al servidor VPS de Bana hosting.

Panel de control de mejoras: Poder controlar todo lo anterior (activar el Captcha, ver las IPs bloqueadas...) directamente desde el panel de administración actual.

5.4 Aviso Legal y RGPD

Aunque todos los puntos están preparados link de Aviso Legal, Política de Privacidad y todo lo referente a ellos, no han sido redactados por carecer de datos reales, esta es una de las partes mas importantes del proyecto porque de ello depende el recibir reclamaciones y/o sanciones ya que trabajamos con datos muy sensibles de nuestros clientes.

6 Bibliografía

Para el desarrollo de Lucina se han consultado diversos recursos técnicos, manuales oficiales y documentación de soporte que han permitido implementar la lógica de IA y las comunicaciones seguras:

6.1 Documentación Técnica y Manuales

Manual Oficial de PHP: Ha sido la fuente principal para la gestión de arrays multidimensionales, el uso de cURL para conectar con la IA de Google y la implementación de tipos en PHP 8.2. (php.net)

Referencia de MariaDB: Utilizada para el diseño del modelo entidad-relación y la optimización de las consultas SQL bajo el motor InnoDB. (mariadb.com)

Configuración de Apache: Guía de consulta para la gestión de hosts virtuales, permisos de directorios en el servidor y configuración de archivos .htaccess.

6.2 Recursos Web y Frameworks

Google Gemini API Reference: Documentación técnica fundamental para conectar con el modelo 1.5 Flash, gestionar las claves de API y aplicar técnicas de Prompt Engineering para obtener respuestas en JSON. (ai.google.dev y [chat gemini pro](https://chat.gemini.pro))

Framework Bootstrap 5.3: Manual de componentes y sistema de rejilla (grid) utilizado para que la web sea totalmente responsiva y funcione bien en móviles. (getbootstrap.com)

MDN Web Docs: Recurso imprescindible para el desarrollo del **Frontend**, especialmente para el uso de la Fetch API (llamadas asíncronas) y la manipulación del DOM con JavaScript moderno. (developer.mozilla.org)

StackOverflow: Consultas puntuales a la comunidad para la resolución de errores lógicos y depuración de bugs durante la fase de integración.

W3Schools: Tutoriales de apoyo para estándares de HTML5, CSS3 y estructuras básicas de bases de datos relacionales.



LUCINA

El mejor comparador de Electricidad